The SIEM & Log Analysis Playbook

A complete, detailed guide to security monitoring, log analysis, and detection engineering with Splunk and Wazuh

From raw log ingestion to tuned detection rules, this playbook builds the query fluency, architecture intuition, and triage workflow that SOC analysts rely on to find real threats in noisy data.

Author: Swastik Sagar
Final-year Computer Science Undergraduate
SOC Analyst Intern

August 1, 2026

Preface

A SIEM is only as useful as the analyst reading its output. Centralizing logs solves a storage problem; turning that pile of events into a detection that actually fires on the right thing, at the right time, without drowning everyone in false positives, is a completely different skill — and it's the one this playbook is about.

This book covers SIEM concepts platform-agnostically first — architecture, data flow, normalization — then goes deep on the two platforms most SOC analysts encounter early in their career: Splunk, the dominant commercial SIEM, and Wazuh, the leading open-source SIEM/XDR. Both get a full query language reference, real detection use cases, and a practical alert-triage workflow, because knowing the syntax is only half the job — knowing what to look for and how to investigate it is the other half.

What this book covers

- SIEM architecture and how data flows from source to alert
- Log sources, collection methods, and normalization (CIM)
- Splunk fundamentals and the Search Processing Language (SPL) in depth
- Splunk dashboards, alerts, and scheduled searches
- Wazuh architecture, agents, and the manager/indexer/dashboard stack
- Writing and tuning Wazuh detection rules and decoders
- Detection engineering fundamentals and the detection lifecycle
- Common detection use cases: brute force, lateral movement, exfiltration, beaconing
- Alert triage and investigation methodology
- Reducing false positives and tuning a noisy SIEM
- Practical query recipes for both platforms

Swastik Sagar

Contents

1

Introduction to SIEM

1.1 What a SIEM actually does

A Security Information and Event Management platform does two jobs at once, which is where its name comes from:

- **Security Information Management (SIM)** — long-term storage, indexing, and search across log data from many sources
- **Security Event Management (SEM)** — real-time correlation, alerting, and dashboards on that data as it arrives

In practice, a SIEM is the place a SOC analyst spends most of their working day: searching historical logs during an investigation, and triaging alerts the platform generated automatically from correlation rules.

1.2 Why centralize logs at all

- **Correlation** — a single failed login means nothing; a hundred failed logins across ten hosts in two minutes, followed by one success, is a brute-force compromise, and only visible with everything in one place
- **Retention** — endpoints rotate their local logs in hours or days; a SIEM can retain data for months, which incident response and compliance both need
- **Search speed** — indexed, centralized logs can be searched in seconds across millions of events, versus SSHing into every host individually
- **Single pane of glass** — one interface instead of dozens of different tools and consoles per log source

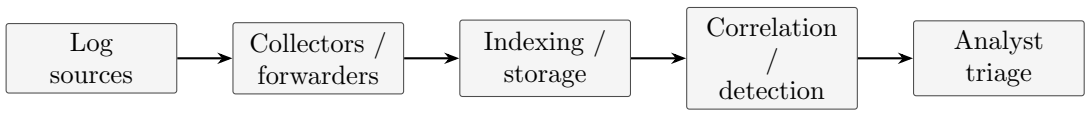

Figure 1.1. *The general SIEM data pipeline, from raw log source to a human making a decision.*

1.3 Common log sources

Source	**Typical value**
Endpoints (Windows/Linux)	Authentication events, process execution, file changes
Firewalls	Allowed/denied connections, NAT translations
DNS servers	Domain resolution requests, useful for detecting beaconing and tunneling
Proxies	Outbound web traffic, URLs visited, blocked categories
Cloud platforms	API calls, IAM changes, resource provisioning (e.g. AWS CloudTrail)
IDS/IPS	Signature and anomaly-based network alerts
Applications	Web server logs, database audit logs, custom app logs

2

SIEM Architecture and Data Flow

2.1 Core components

- **Agents / forwarders** — lightweight software on the source host that ships logs onward
- **Collectors / ingestion layer** — receives, parses, and often normalizes incoming data
- **Indexer / storage layer** — stores logs in a searchable, usually time-series-optimized format
- **Search head / query layer** — where analysts run searches and build dashboards
- **Correlation engine** — runs rules continuously against incoming data to generate alerts

2.2 Push vs. pull collection

- **Push (agent-based)** — an installed agent actively sends logs to the SIEM; used by Wazuh agents and Splunk universal forwarders
- **Pull (agentless)** — the SIEM polls the source, e.g. querying a syslog server, an API, or a cloud provider's logging service

2.3 Log normalization

Raw logs from different vendors describe the same event completely differently — a firewall and an EDR both log "a connection happened" using totally different field names. Normalization maps these into a common schema so a single search can cover every source at once.

Splunk's Common Information Model

Splunk's CIM is a set of prebuilt field-name standards (e.g. `src`, `dest`, `action`) that vendor-specific Splunk apps map their raw fields onto, so a search written against the CIM works across many different log sources without modification.

2.4 Data flow example: a failed login

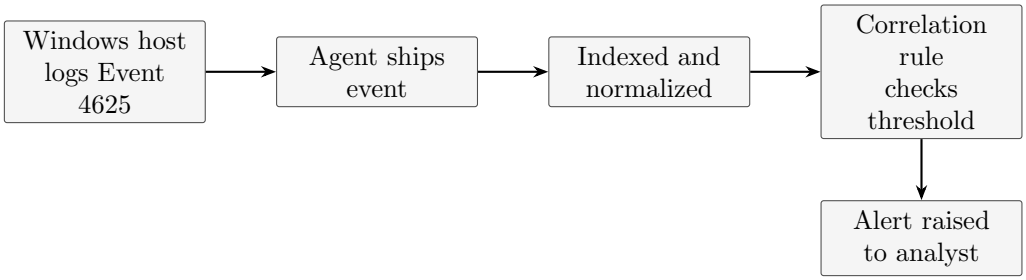

Figure 2.1. *One failed Windows login (Event ID 4625) traced through the full pipeline to an alert.*

3

Splunk Fundamentals

3.1 Splunk architecture

Component	Role
Universal Forwarder	Lightweight agent installed on a source host, ships raw data
Heavy Forwarder	Can parse and filter data before forwarding, more resource-intensive
Indexer	Stores and indexes incoming data, handles search execution
Search Head	Where users run searches and view dashboards; can be separate from the indexer in larger deployments
Deployment Server	Centrally manages configuration across many forwarders

3.2 Indexes, sourcetypes, and events

- An **index** is where Splunk physically stores data, typically segmented by data type or retention need (e.g. `index=firewall`)
- A **sourcetype** tells Splunk how to parse a given format (e.g. `sourcetype=WinEventLog`)
- An **event** is a single timestamped record — one log line, normally

Basic search syntax

```
index=firewall sourcetype=cisco_asa dest_port=445
```

Search efficiency

Always filter by index and time range first, before adding other terms — Splunk searches narrow from the earliest terms onward, so front-loading selective filters (index, sourcetype, a specific field) dramatically speeds up large searches compared to filtering at the end.

4

Splunk Search Processing Language (SPL)

4.1 The pipeline model

SPL works like a Unix pipeline: each command after a pipe (`|`) takes the output of the previous command as its input, transforming it further.

Anatomy of an SPL search

```
index=auth sourcetype=linux_secure "Failed password" | stats count by src_ip |
where count > 20 | sort -count
```

4.2 Core commands

Command	Purpose
`stats`	Aggregate results: `count`, `sum`, `avg`, `dc` (distinct count), grouped by one or more fields
`table`	Display only specified fields, in order
`where`	Filter results with a boolean expression, evaluated after the field exists
`eval`	Create or modify a field using an expression
`rex`	Extract a new field using a regular expression
`sort`	Order results by one or more fields; prefix with `-` for descending
`dedup`	Remove duplicate events based on specified fields
`timechart`	Aggregate results over time buckets, ideal for trend graphs
`transaction`	Group related events into a single transaction by shared field(s) and time proximity
`lookup`	Enrich events with data from an external CSV or KV store

4.3 Useful search examples

Top talkers by bytes transferred

```
index=firewall | stats sum(bytes) as total_bytes by src_ip | sort -total_bytes |
head 10
```

Detecting a brute-force pattern

```
index=auth "Failed password" | bin _time span=5m | stats count by src_ip, _time |
where count > 15
```

Extracting a field with rex

```
index=web | rex field=_raw "user=(?<username>\w+)" | stats count by username
```

eval vs. where

`eval` creates or transforms a field; `where` filters events based on a condition. A common pattern is `eval` to build a field, then `where` to filter on it in the next pipeline stage.

4.4 Time modifiers

Common time range syntax

```
earliest=-24h latest=now earliest=-7d@d latest=@d earliest=0 latest=now
```

5

Splunk Dashboards and Alerts

5.1 Building a dashboard panel

Dashboards are built from saved searches rendered as visualizations — tables, line charts, single-value panels, and maps are the most common. Every panel is, underneath, just an SPL search with a `timechart`, `stats`, or `geostats` command feeding a chart type.

A panel-ready search

```
index=firewall action=blocked | timechart span=1h count by action
```

5.2 Alerts

An alert is a saved search that runs on a schedule (or in real time) and triggers an action — an email, a webhook, or adding to Splunk's own Notable Events index — when its trigger condition is met.

Trigger type	Behavior
Number of results	Fires when the search returns more (or fewer) than a threshold
Number of hosts	Fires based on distinct host count in results
Custom condition	Fires based on a custom SPL condition evaluated against results
Real-time	Evaluates continuously rather than on a schedule

Alert fatigue

An alert that fires constantly gets ignored, which is worse than not having it at all. Tune thresholds against real baseline data before deploying an alert broadly, and prefer fewer, well-tuned alerts over many noisy ones.

6

Wazuh Fundamentals

6.1 Wazuh architecture

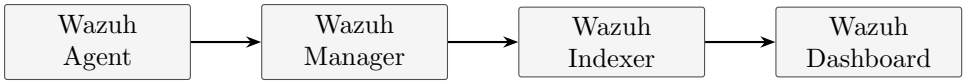

Figure 6.1. *Wazuh's four-part architecture: agents collect, the manager analyzes and rules-matches, the indexer stores, the dashboard visualizes.*

Component	Role
Agent	Installed on monitored endpoints; collects logs, file integrity data, and runs vulnerability/config checks
Manager	Receives agent data, applies decoders and rules, generates alerts
Indexer	OpenSearch-based storage and search backend for alert data
Dashboard	Web UI (Kibana-based) for browsing, searching, and visualizing alerts

6.2 Core Wazuh capabilities

- **Log data analysis** — collects and analyzes logs from agents and syslog sources
- **File Integrity Monitoring (FIM)** — detects changes to monitored files and directories
- **Vulnerability detection** — correlates installed software against known CVE feeds
- **Security configuration assessment (SCA)** — checks systems against hardening benchmarks
- **Active response** — can automatically execute a script in reaction to an alert (e.g. blocking an IP)

6.3 Installing an agent

Linux agent install and registration

```
sudo apt install wazuh-agent sudo systemctl enable wazuh-agent sudo systemctl
start wazuh-agent -- WAZUH_MANAGER IP is normally set during install or via --
/var/ossec/etc/ossec.conf
```

7

Wazuh Rules and Decoders

7.1 Decoders

A decoder tells Wazuh how to parse a raw log line into structured fields — extracting things like source IP, username, or status code from otherwise unstructured text, so rules can match against specific fields rather than raw strings.

Simplified custom decoder, decoder.xml

```
<decoder name="my-app-login"> <program_name>myapp</program_name> </decoder>
<decoder name="my-app-login"> <parent>my-app-login</parent> <regex>user (\S+)
login (\w+) from (\S+)</regex> <order>user, status, srcip</order> </decoder>
```

7.2 Rules

Rules match against decoded fields (or raw log text) and assign a severity level; rules can also reference other rules to build multi-stage detection logic.

Simplified custom rule, local_rules.xml

```
<rule id="100001" level="5"> <decoded_as>my-app-login</decoded_as>
<field name="status">failed</field> <description>My-app failed login
attempt</description> </rule>
<rule id="100002" level="10" frequency="8" timeframe="120">
<if_matched_sid>100001</if_matched_sid> <description>Possible brute force against
my-app</description> </rule>
```

Rule levels

Wazuh rule severity runs 0–16. Levels 0–3 are typically informational, 4–7 are low-to-medium, 8–11 are notable security events worth reviewing, and 12+ are high-severity, often correlation-based rules like the brute-force example above.

7.3 Testing rules before deployment

Test a log line against current rules/decoders

```
sudo /var/ossec/bin/wazuh-logtest
```

8

Detection Engineering Fundamentals

8.1 The detection lifecycle

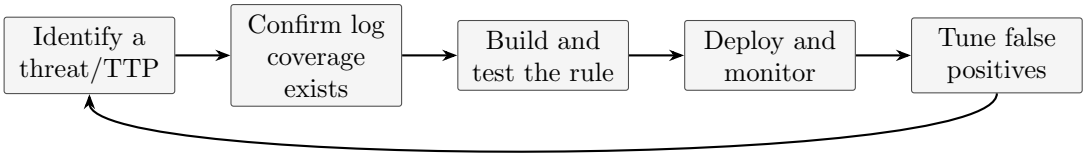

Figure 8.1. *Detection engineering is a loop, not a one-time task – tuning feeds back into refining the original idea.*

8.2 Detection quality: precision vs. recall

- **High recall, low precision** — catches almost every true positive, but drowns analysts in false alarms
- **High precision, low recall** — rarely wrong when it fires, but misses real attacks that don't exactly match the pattern
- Good detection engineering deliberately balances the two based on the severity of what's being detected — a ransomware-execution rule should favor recall; a routine-policy-violation rule should favor precision

8.3 Where detections come from

- **Threat intelligence** — known IOCs (IPs, hashes, domains) tied to active campaigns
- **MITRE ATT&CK** — building detections mapped to specific adversary techniques
- **Purple teaming** — red team executes a technique, blue team confirms it was (or wasn't) detected
- **Incident post-mortems** — writing a detection for a gap found during a real investigation

9

Common Detection Use Cases

9.1 Brute force / credential stuffing

- Multiple failed authentications from one source in a short window
- Failed logins across many different accounts from one source (spraying)
- A failure immediately followed by a success from the same source

9.2 Lateral movement

- Unusual authentication from a workstation directly to another workstation, rather than to a server
- A single account authenticating to an unusually high number of distinct hosts in a short period
- Use of admin tools (PsExec, WMI, RDP) from a host that doesn't normally use them

9.3 Data exfiltration

- Large outbound data transfer to an external or previously unseen destination
- Transfers to cloud storage or paste sites outside normal business use
- Off-hours transfers significantly larger than a host's historical baseline

9.4 C2 beaconing

- Highly regular, periodic outbound connections at a fixed interval
- DNS queries to newly registered or algorithmically generated domains (DGA)
- Consistent small-byte-count connections to the same external host over time

Beaconing detection intuition

Legitimate traffic is bursty and irregular; beaconing malware calling home every 60 seconds looks unnaturally regular in a timechart — a suspiciously flat, evenly-spaced pattern is often more suspicious than a spike.

10

Alert Triage and Investigation

10.1 A repeatable triage workflow

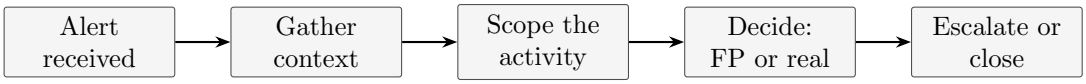

Figure 10.1. *A consistent triage sequence keeps investigations fast and defensible, especially under alert volume.*

10.2 Questions to answer during triage

- What exactly triggered this alert, in plain terms?
- Is the source host or account expected to behave this way?
- What happened immediately before and after, on the same host?
- Has this same pattern occurred before, and was it resolved as benign?
- If real, what is the blast radius — one host, one account, or wider?

10.3 Documentation matters

Every closed alert should record what was checked and why the verdict was reached, even for false positives — this is what lets tuning happen later, and it's what protects the analyst if the same alert turns out to matter in hindsight.

Pivoting from one alert

The alert itself is rarely the whole story. Pivoting on the source IP, account, or host into surrounding logs — not just the exact matched event — is usually what separates a real finding from a five-second false-positive close.

11

Tuning and Reducing False Positives

11.1 Why tuning matters

An untuned SIEM trains analysts to ignore alerts, which is the single most dangerous outcome a detection program can produce — a real intrusion buried in noise is functionally the same as no detection at all.

11.2 Tuning techniques

Technique	When to use it
Allowlisting	A specific, known-good host/account/process repeatedly triggers a legitimate rule
Threshold adjustment	The rule is directionally correct but fires too easily; raise the count or shrink the time window
Additional conditions	Add a field constraint that separates the false-positive pattern from the true-positive one
Suppression windows	Same alert on the same asset shouldn't re-fire for X minutes once acknowledged
Rule retirement	The detection no longer maps to a credible threat and produces only noise

Never silently disable

Retiring or suppressing a rule should be a deliberate, documented decision — not a quiet workaround for alert fatigue. An undocumented suppression is a detection gap nobody knows exists until it's needed.

12

Practical Query Recipes

12.1 Splunk: failed then successful login (possible brute force success)

SPL

```
index=auth (action=failure OR action=success) | transaction user maxspan=10m |
where mvcount(action)>1 AND searchmatch("failure") AND searchmatch("success")
```

12.2 Splunk: rare processes by host

SPL

```
index=endpoint sourcetype=sysmon EventCode=1 | stats count by host, process_name |
where count < 3
```

12.3 Splunk: DNS query volume spike per host

SPL

```
index=dns | bin _time span=1h | stats dc(query) as unique_queries by host, _time |
where unique_queries > 500
```

12.4 Wazuh: searching alerts by rule level

Dashboard query / KQL-style filter

```
rule.level:>=10
```

12.5 Wazuh: alerts for a specific agent

Dashboard query

```
agent.name:"web-server-01" AND rule.level:>=7
```

12.6 Wazuh: File Integrity Monitoring alerts

Dashboard query

```
rule.groups:"syscheck" AND syscheck.event:"added"
```